

TRƯỜNG ĐẠI HỌC XÂY DỰNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO THỰC HÀNH BÀI TẬP LỚN
NHẬP MÔN TRÍ TUỆ NHÂN TẠO
Đề tài: MÔ HÌNH LOGISTIC REGRESSION
ỨNG DỤNG ĐỂ PHÂN LỚP CHO BỘ DỮ LIỆU MNIST

Nhóm 8:

Nhóm sinh viên: Bùi Hải Anh

Đinh Ngọc Ca

Lê Thành Công

Lớp: 64CS2

Giảng viên: Nguyễn Đình Quý

HÀ NỘI 06-2021

MỤC LỤC

I, Lời nói đầu.....	2
II, CƠ SỞ DỮ LIỆU MNIST.....	3
III, NỘI DUNG.....	4
1. LÝ THUYẾT LOGISTIC REGRESSION.....	4
1. Định nghĩa logistic regression.....	4
2. Xây dựng model.....	4
3. Sigmoid function.....	6
4. Loss function.....	7
5. Softmax function.....	7
2. BÁO CÁO CÁC PART:.....	8
Part 1.....	8
Part 2.....	8
Part 3.....	9
Part 3.....	9

6

I, Lời nói đầu

Đây là bản báo cáo dùng để biểu diễn thiết kế và giải quyết bài toán phân lớp cho bộ dữ liệu MNIST sử dụng mô hình logistic regression và thuật toán học gradient descent do nhóm em thiết kế và lập trình. Tài liệu này giúp ta có cái nhìn rõ hơn về cơ sở dữ liệu MNIST cũng như ứng dụng mô hình logistic regression và thuật toán gradient descent để phân lớp cho bộ dữ liệu này. Nhóm thực hiện đề tài nhằm mục đích xây dựng và đánh giá độ dốc gradient để giảm thiểu rủi ro theo thực nghiệm trong một mô hình đa thức logistic regression trên tập dữ liệu MNIST. Trong quá trình thực hiện đề tài không tránh khỏi những sai sót, nhóm em mong sẽ nhận được sự góp ý và đánh giá của thầy.

Xin chân thành cảm ơn !

II, CƠ SỞ DỮ LIỆU MNIST

Cơ sở dữ liệu MNIST là một cơ sở dữ liệu lớn gồm các chữ số viết tay thường được sử dụng để đào tạo các hệ thống xử lý ảnh khác nhau. (download tại <http://yann.lecun.com/exdb/mnist/>), có bộ đào tạo gồm 60.000 ví dụ và bộ kiểm tra gồm 10.000 ví dụ. Nó là một tập hợp con của một tập hợp lớn hơn có sẵn từ NIST. Các chữ số đã được chuẩn hóa kích thước và căn giữa trong một hình ảnh có kích thước cố định. Hơn nữa, các hình ảnh đen trắng từ NIST đã được chuẩn hóa để vừa với hộp giới hạn 28x28 pixel và khử răng cưa, điều này đưa ra các mức thang độ xám.

Cơ sở dữ liệu cũng được sử dụng rộng rãi để đào tạo và thử nghiệm trong lĩnh vực học máy. Nó được tạo ra bằng cách "trộn lại" các mẫu từ bộ dữ liệu ban đầu của NIST. Đây là một cơ sở dữ liệu tốt cho những người muốn thử học các kỹ thuật và phương pháp nhận dạng mẫu trên dữ liệu thể giới thực trong khi dành ít nỗ lực cho việc tiền xử lý và định dạng.

III, NỘI DUNG

1. Định nghĩa logistic regression

- Hồi quy logistic là một phương pháp phân tích thống kê được sử dụng để dự đoán giá trị dữ liệu dựa trên các quan sát trước đó của tập dữ liệu.

- Mục đích của hồi quy logistic là ước tính xác suất của các sự kiện, bao gồm xác định mối quan hệ giữa các tính năng từ đó dự đoán xác suất của các kết quả, nên đối với hồi quy logistic ta sẽ có:

+ Input: dữ liệu input (ta sẽ coi có hai nhãn là 0 và 1).

+ Output : Xác suất dữ liệu input rơi vào nhãn 0 hoặc nhãn 1.

2. Xây dựng model

- Xét trên một điểm dữ liệu ta có input $x=[x_1;x_2;\dots;x_n]$ sẽ là một vector cột, ta sẽ ngăn cách các feature(x_i) bằng dấu “,”. input x sẽ có dạng như bên dưới:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

- Xét trên toàn bộ dữ liệu (m điểm dữ liệu) ta sẽ có một vector hàng $X=[x(1),x(2),\dots,x(m)]$ với mỗi cột là một điểm dữ liệu x_i ngăn cách nhau bởi dấu “,”:

$$X = [x^{(1)}, x^{(2)}, \dots, x^{(m)}] = \textcolor{red}{\text{X}}$$

Ta có thể thấy X có được bằng cách xếp thành cột các các $x(i)$.

- Bộ trọng số $w=[w_1;w_2;\dots;w_n]$ cũng sẽ là một vector cột, ta sẽ ngăn cách các trọng số w_i bằng dấu “;” bộ trọng số w sẽ có dạng như dưới:

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

- Và ta có chuyển vị của vector w sẽ là một vector hàng, các w_i ngăn cách nhau bằng dấu “,” :

$$w^T = [w_1, w_2, \dots, w_n]$$

- Xét trên một điểm dữ liệu ta đặt $z=w^T x+b$ (b gọi là bias), và $a=\sigma(z)$:

$$z = w^T x + b$$

$$a = \sigma(z)$$

- Xét trên toàn bộ tập dữ liệu ta đặt $Z=[z(1),z(2),\dots,z(m)]$ và $A=[a(1),a(2),\dots,a(m)]$, đây đều là các vector hàng với m là số điểm dữ liệu:

$$Z = [z^{(1)}, z^{(2)}, \dots, z^{(m)}]$$

$$A = [a^{(1)}, a^{(2)}, \dots, a^{(m)}]$$

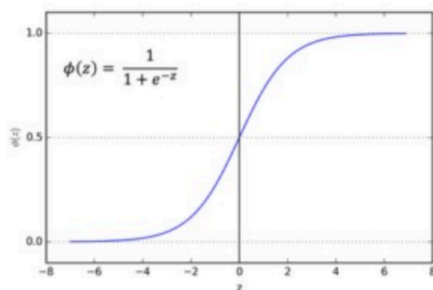
Note: Việc kí hiệu x, w là vector cột và X có được bằng cách xếp các $x(i)$ thành cột là để thuận lợi trong việc tính toán và triển khai code, cách kí hiệu là không bắt buộc nghĩa sao việc tính toán được thuận lợi là được.

3. Sigmoid function

Hàm Sigmoid (hay còn gọi là hàm số Logistic) là một hàm số toán học có đường cong dạng hình chữ “S” với công thức như sau:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Mọi giá trị khi đi qua hàm Sigmoid sẽ nằm trong miền giá trị số thực chạy từ 0.0 đến 1.0, Đồ thị hàm sigmoid:



Nguồn: [quora.com/What-does-an-x-axis-represent-in-a-logistic-regression-sigmoid-function](https://www.quora.com/What-does-an-x-axis-represent-in-a-logistic-regression-sigmoid-function)

Nhìn vào đồ thị trên ta có thể thấy giá trị khi đi qua hàm sigmoid sẽ tiệm cận đến 1 khi đầu vào tiến đến ∞ và sẽ tiệm cận đến 0 khi đầu vào tiến đến $-\infty$ và sẽ bằng 0.5 khi $x=0$.

Nhờ vào đặc tính này mà hàm sigmoid được sử dụng nhiều trong lĩnh vực trí tuệ nhân tạo với vai trò là hàm kích hoạt (hàm kích hoạt giúp định nghĩa đầu ra khi có một input đi qua nó).

4. Loss function

Hàm mất mát trong thuật toán học có giám sát(supervised learning) nói chung là hàm số để đo sự sai khác giữa giá trị thực tế và giá trị mô hình dự đoán, sự sai khác có thể sai khác về giá trị thực(như trong hồi quy tuyến tính) hoặc sai khác về nhãn(như trong bài toán phân loại) việc định nghĩa sự sai khác như nào là tùy vào từng bài toán cụ thể để từ đó xây dựng hàm mất mát.

Hàm mất mát của hồi quy logistic, hàm này còn có tên gọi là Cross-Entropy:

$$L(w) = - \sum_{i=0}^m \hat{y}_i \log y_i$$

5. Softmax function

Chúng ta cần một mô hình xác suất sao cho với mỗi input x , a_i thể hiện xác suất để input đó rơi vào class i , vậy điều kiện cần là các a_i phải dương và tổng của chúng bằng 1. Để có thể thỏa mãn điều kiện này, chúng ta cần nhìn vào mọi giá trị z_i và dựa trên quan hệ giữa các z_i này để tính toán giá trị của a_i . Ngoài các điều kiện a_i lớn hơn 0 và có tổng bằng 1, chúng ta sẽ thêm một điều kiện cũng rất tự nhiên nữa, đó là: giá trị $z_i = w_i^T X$ càng lớn thì xác suất dữ liệu rơi vào class i càng cao. Điều kiện cuối này chỉ ra rằng chúng ta cần một hàm đồng biến ở đây.

Hàm softmax có phương trình

$$a_i = \frac{\exp(z_i)}{\sum_{j=1}^C \exp(z_j)}$$

2. Báo cáo các part:

Part 1:

Sử dụng loss function để xác định mức độ sai khác của kết quả dự đoán so với giá thực thực cần dự đoán.

Ta dùng hàm softmax để chuyển các giá trị sang xác suất

$$L(X_i, Y_i) = - \sum_{j=1}^c y_{ij} * \log(p_{ij})$$

where Y_i is one-hot encoded target vector $(y_{i1}, y_{i2}, \dots, y_{ic})$,

$$y_{ij} = \begin{cases} 1, & \text{if } i_{th} \text{ element is in class } j \\ 0, & \text{otherwise} \end{cases}$$

$p_{ij} = f(X_i) = \text{Probability that } i_{th} \text{ element is in class } j$

$$\nabla_w L(h_w(x), y) = (\text{softmax}(Wx) - y)x^T$$

Từ đó tìm ra được biểu thức tính tổng tất cả cross-entropy trên toàn bộ training data:

$$-\log L(\{y^{(n)}\}, \{\hat{y}^{(n)}\}) = \sum_n \left[- \sum_i \log(\hat{y}^{(n)}_i) \right] = \sum_n H(y^{(n)}), \text{ i.e.}$$

Đến đây chúng ta sử dụng hàm `test_gradient` để kiểm tra xem hàm `multinomial_logreg_grad_i` có thực sự là gradient của `multinomial_logreg_loss_i` không?

Kết quả test:

Đối với $\alpha = \{0, 1\}$, chênh lệch trung bình là: $\{0, 342\}$

Đối với $\alpha = \{0, 001\}$, chênh lệch trung bình là: $\{0, 003\}$

Đối với $\alpha = \{1e-05\}$, chênh lệch trung bình là: $\{3, 426\}$

```
Running part 1 ...

For alpha=0.1, average difference is: 0.34104167318046186
For alpha=0.001, average difference is: 0.0034356638345701624
For alpha=1e-05, average difference is: 3.427686717568636e-05
```

Part 2:

- Hàm `multinomial_logreg_total_grad` tính tổng độ lỗi của dữ liệu

Chạy thuật toán với $\alpha=1.0$, $\gamma=0.0001$ ta được

Thời gian chạy hết thuật toán sau 10 lần lặp là 2'3s

```
Running part 2 ...  
Running starter code implementation config: alpha=1.0, gamma=0.0001, #iterations=10, monitorFreq=10  
Time taken for the above config is: 0:02:03.116228  
#####
```

Part 3:

Dùng hàm `multinomial_logreg_total_loss` để tính tổng hàm loss

Thời gian chạy hết thuật toán sau 10 lần lặp là 0:00:03.6

Ta thấy cách tiếp cận của PART 3 nhanh hơn so với PART 2 bởi vì ở PART 3 ta sử dụng ma trận Numpy số học thay vì các vòng lặp trong python.

```
Running part 3 ...  
Running numpy implementation config: alpha=1.0, gamma=0.0001, #iterations=10, monitorFreq=10  
Time taken for the above config is: 0:00:03.573145
```

Part 4:

Thời gian chạy: 0:02:29

Thời gian chạy để nhận được lỗi đào tạo với toàn bộ tập dữ liệu trên một mô hình là: 0:00:02

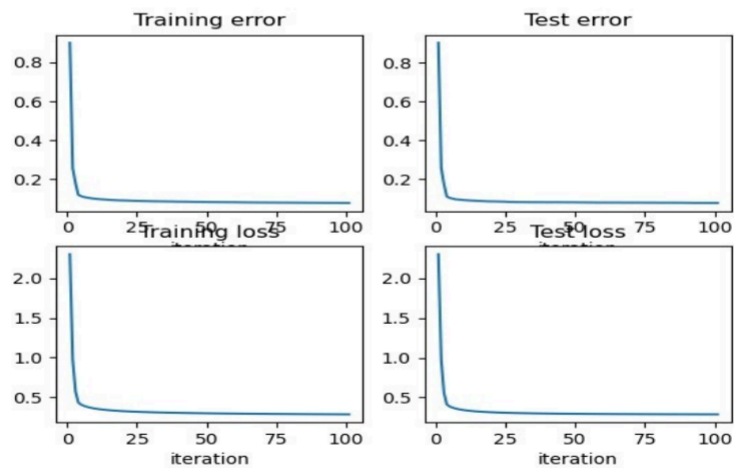
Thời gian chạy để nhận được lỗi huấn luyện với việc lấy 100 mẫu con trên một mô hình là: 0:00:00.02

Thời gian chạy để nhận được lỗi huấn luyện với việc lấy 1000 mẫu con trên một mô hình là: 0:00:00.024

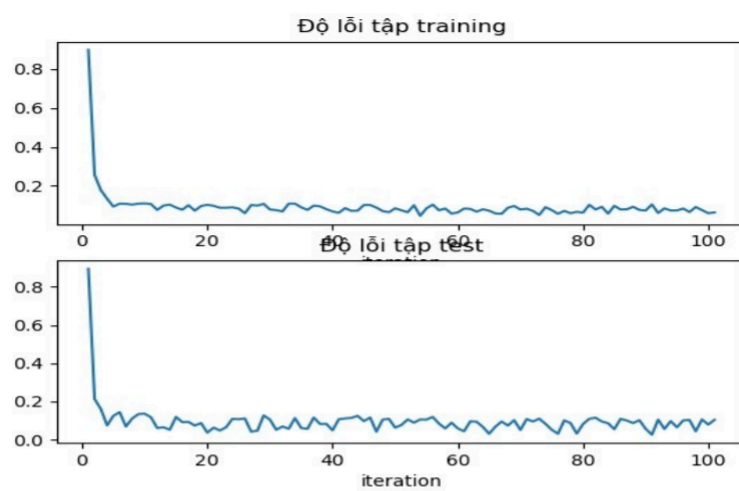
```
Running part 4 ...  
  
Running numpy implementation config: alpha=1.0, gamma=0.0001, #iterations=1000, monitorFreq=10  
Time taken for the above config is: 0:02:29.648870  
  
Running time to obtain training error with entire dataset on a single model is: 0:00:02.716214  
  
Running time to obtain training error with subsampling 100 samples on a single model is: 0:00:00.024089  
  
Running time to obtain training error with subsampling 1000 samples on a single model is: 0:00:00.065821
```

Evaluating gradient descent

-Vẽ đồ thị:



-So sánh thời gian chạy của các đánh giá lỗi này :



Hà Nội, ngày 05 tháng 06 năm 2021

Thành viên nhóm 8:

Bùi Hải Anh

Đinh Ngọc Ca

Lê Thành Công